

# ООП 2

ОСНОВНИ ПРИНЦИПИ И ТЕХНИКИ В ООП (ООД)

# ПРИНЦИПИ НА ОБЕКТНО-ОРИЕНТИРАНИЯ ДИЗАЙН

- КАКВО Е СОФТУЕРНА АРХИТЕКТУРА?
  - ПИРАМИДА ОТ СЛОЕВЕ
    - SOA, MICROSERVICE.....REST
    - МОДУЛИ И ТЕХНИТЕ ВРЪЗКИ
      - ПАКЕТИ, КОМПОНЕНТИ И КЛАСОВЕ

# ПРИНЦИПИ НА ООП ДИЗАЙН

- ЗАЩО Е ВАЖНА АРХИТЕКТУРАТА?
  - ЛОША АРХИТЕКТУРА -> ОСНОВЕН ПРИНОС ЗА НАРАСТВАНЕТО НА УЯЗВИМОСТТА НА СОФТУЕРА
  - ТРУДЕН ЗА ЧЕТЕНЕ КОД
  - ТРУДЕН ЗА ПОДДЪРЖАНЕ
    - ТРУДНА МОДИФИКАЦИЯ
    - ПО-БАВНА ИМПЛЕМЕНТАЦИЯ НА НОВИ ФУНКЦИОНАЛНОСТИ
    - ПРЕДПОСТАВКИ ЗА ПОВЕЧЕ ДЕФЕКТИ

# ПРИНЦИПИ НА ООП ДИЗАЙН

- СИМПТОМИ НА ЛОША АРХИТЕКТУРА
  - ТВЪРДОСТ - RIGIDITY
  - ЧУПЛИВОСТ - FRAGILITY
  - НЕПОДВИЖНОСТ - IMMOBILITY
  - ВИСКОЗИТЕТ - VISCOSITY
  - НЕНУЖНА СЛОЖНОСТ - NEEDLESS COMPLEXITY
  - НЕНУЖНО ПОВТОРЕНИЕ - NEEDLESS REPETITION
  - НЕПРОЗРАЧНОСТ - OPACITY

# ТВЪРДОСТ (RIGIDITY) НА ДИЗАЙН

- ТЕНДЕНЦИЯТА СОФТУЕРЪТ ДА БЪДЕ ТРУДЕН ЗА ПРОМЯНА
  - ДОРИ ЗА ДРЕБНИ НЕЩА
- ВСЯКА ПРОМЯНА ПРИЧИНЯВА КАСКАДА ОТ ПОСЛЕДВАЩИ ПРОМЕНИ В ЗАВИСИМИ МОДУЛИ
- ОКАЗВА ЛОШО ВЛИЯНИЕ НА УПРАВЛЕНИЕТО НА ПРОЕКТА
- “РАБОТИ НЕ ГО ПИПАЙ”

# ЧУПЛИВОСТ (FRAGILITY) НА ДИЗАЙН

- ТЕНДЕНЦИЯТА НА СОФТУЕР ДА СЕ СЧУПИ НА МНОГО МЕСТА ПРИ ВСЯКА ПРОМЯНА
  - ЧЕСТО СЧУПВАНЕТО СЕ СЛУЧВА В ОБЛАСТИ, КОИТО НЯМАТ КОНЦЕПТУАЛНА ВРЪЗКА С ПРОМЕНЕНАТА ОБЛАСТ
  - ВЕРОЯТНОСТТА ОТ СЧУПВАНЕ СЕ УВЕЛИЧАВА С ВРЕМЕТО
    - „РАЗРАБОТЧИЦИТЕ СА ЗАГУБИЛИ КОНТРОЛ НАД СОФТУЕРА СИ“

# НЕПОДВИЖНОСТ (IMMOBILITY) НА ДИЗАЙН

- НЕВЪЗМОЖНОСТТА ЗА ПОВТОРНО ИЗПОЛЗВАНЕ НА СОФТУЕР ОТ ДРУГИ ПРОЕКТИ ИЛИ ОТ СЪЩИЯ ПРОЕКТ
  - НЕОБХОДИМОСТ ОТ ПРЕНАПИСВАНЕ ВМЕСТО ПРЕИЗПОЛЗВАНЕ

# ВИСКОЗИТЕТ (VISCOSITY) НА ДИЗАЙН

- ВИСКОЗИТЕТ НА ДИЗАЙНА И ВИСКОЗИТЕТ НА СРЕДАТА
- ПРОМЯНА НА ДИЗАЙНА
  - НОРМАЛЕН
  - ХАКОВЕ
  - НАДДЕЛЯВА ПО ЛЕСНОТО
- СРЕДАТА ЗА РАЗРАБОТКА Е БАВНА И НЕЕФЕКТИВНА
  - КОМПИЛИРАНЕ Е МНОГО ДЪЛГО
  - СИСТЕМАТА ЗА КОНТРОЛ НА ИЗХОДНИЯ КОД ИЗИСКВА ЧАСОВЕ ЗА ПРОВЕРКА САМО НА НЯКОЛКО ФАЙЛА

# СИМПТОМИ НА ЛОША АРХИТЕКТУРА

- НЕНУЖНА СЛОЖНОСТ
  - ДИЗАЙНЪТ ВКЛЮЧВА ЕЛЕМЕНТИ, КОИТО В МОМЕНТА НЕ СА ПОЛЕЗНИ. МОЖЕ БИ ДИЗАЙНЕРЪТ ОЧАКВА ТЕ ДА БЪДАТ ПОЛЕЗНИ ПО-КЪСНО...
- НЕНУЖНО ПОВТОРЕНИЕ
  - КАКВО Е ДРУГОТО ИМЕ НА ТОЗИ ПРОБЛЕМ?
- НЕПРОЗРАЧНОСТ
  - ДАДЕН МОДУЛ Е ТРУДЕН ЗА РАЗБИРАНЕ, НЕ Е НАПИСАН ЯСНО.
  - КОДЪТ ИМА ТЕНДЕНЦИЯ ДА СТАВА ВСЕ ПО-НЕПРОЗРАЧЕН С НАПРЕДВАНЕТО НА ВРЕМЕТО, ЗАЩОТО НИКОЙ НЕ Е ЗАПОЗНАТ ОТБЛИЗО С НЕГО ВЕЧЕ

# ПРИЧИНИ ЗА ПРОБЛЕМИ

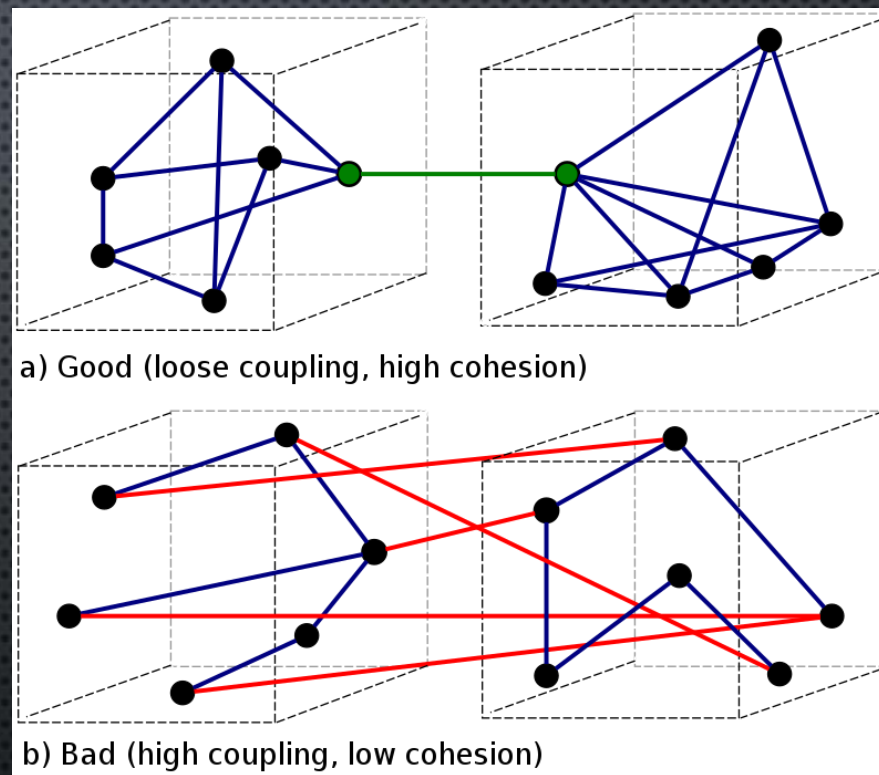
- КАЧЕСТВОТО НА ПРОГРАМИСТИТЕ!
- ИЗИСКВАНИЯТА СЕ ПРОМЕНЯТ ПО НАЧИНИ, КОИТО ПЪРВОНАЧАЛНИЯТ ДИЗАЙН НЕ Е ПРЕДВИДИЛ
- ЧЕСТО ТЕЗИ ПРОМЕНИ ТРЯБВА ДА БЪДАТ НАПРАВЕНИ БЪРЗО И МОГАТ ДА БЪДАТ НАПРАВЕНИ ОТ ИНЖЕНЕРИ, КОИТО НЕ СА ЗАПОЗНАТИ С ОРИГИНАЛНАТА ФИЛОСОФИЯ НА ДИЗАЙНА.
- НАТРУПВАНЕ НА ПРОБЛЕМИ
- ПРОБЛЕМЪТ Е В АРХИТЕКТУРАТА (В НАС)

# ПРИЧИНИ ЗА ПРОБЛЕМИ

- ПРОМЕНИ, КОИТО ВЪВЕЖДАТ НОВИ И НЕПЛАНИРАНИ ЗАВИСИМОСТИ
  - НЕПРАВИЛНИ ЗАВИСИМОСТИ МЕЖДУ МОДУЛИТЕ НА СОФТУЕРА
  - ТРЯБВА ДА СЕ УПРАВЛЯВАТ ЗАВИСИМОСТИТЕ МЕЖДУ МОДУЛИТЕ В ПРИЛОЖЕНИЕТО
  - ИЗГРАЖДАНЕ НА ЗАЩИТНИ СТЕНИ
- 
- ОБЕКТНО-ОРИЕНТИРАНИЯТ ДИЗАЙН Е ПЪЛЕН С ПРИНЦИПИ И ТЕХНИКИ ЗА ИЗГРАЖДАНЕ НА ТАКИВА ЗАЩИТНИ СТЕНИ И ЗА УПРАВЛЕНИЕ НА ЗАВИСИМОСТИТЕ МЕЖДУ МОДУЛИТЕ

# ЦЕЛИ В ООД

- НИСКА ЗАВИСИМОСТ - LOW COUPLING
- СИЛНА КОХЕЗИЯ - HIGH COHESION



# НИСКА ЗАВИСИМОСТ - LOW COUPLING

- ДО КОЛКО РАЗЛИЧНИТЕ КЛАСОВЕ (МОДУЛИ) ЗАВИСЯТ ЕДИН ОТ ДРУГ?
- ПРОБЛЕМИ ПРИ СИЛНА СВЪРЗАНОСТ
- ПРЕДИМСТВА ПРИ НИСКАТА СВЪРЗАНОСТ

# СИЛНА КОХЕЗИЯ - HIGH COHESION

- СВЪРЗАНИЯТ КОД ДА Е НА ЕДНО МЯСТО
  - ПОДПОМАГА DRY
- КОЛКО МОДУЛА СЕ ПРОМЕНЯТ ЗА ДА СЕ ОПРАВИ ПРОБЛЕМ ИЛИ ДА СЕ СЪЗДАДЕ ФУНКЦИОНАЛНОСТ
- НА КОЛКО РАЗЛИЧНИ МЕСТА ТРЯБВА ДА СЕ ИЗВЪРШИ ТАЗИ ПРОМЯНА?
- КОЛКО ТРУДНО Е ДА ТЕСТВАМ МОЯ КОД?

# ПОЛЗИ ОТ ПРАВИЛЕН ООД

- ЛЕСНО ПИСАНЕ
- ЛЕСНО ТЕСТВАНЕ
- ЛЕСНО ПОДДЪРЖАНЕ

# ПРИНЦИПИ НА ООД

- S.O.L.I.D !!!
  - SINGLE RESPONSIBILITY PRINCIPLE
  - OPEN CLOSED PRINCIPLE
  - LISKOV SUBSTITUTION PRINCIPLE
  - INTERFACE SEGREGATION PRINCIPLE
  - DEPENDENCY INVERSION PRINCIPLE

# SINGLE RESPONSIBILITY PRINCIPLE (SRP)

## ПРИНЦИП ЗА ЕДИНСТВЕНА ОТГОВОРНОСТ

- КЛАСА ТРЯБВА ДА ИМА САМО ЕДНА ОТГОВОРНОСТ
  - ...САМО ЕДНА ПРИЧИНА ДА СЕ ПРОМЕНЯ

# SINGLE RESPONSIBILITY PRINCIPLE (SRP)

## ПРИНЦИП ЗА ЕДИНСТВЕНА ОТГОВОРНОСТ

### ПРИМЕР...

- CITYMAP
  - СПИСЪК С ГРАДОВЕ
  - ИЗЧИСЛЯВАНЕ НА НАСЕЛЕНИЕ
  - ИЗЧЕРТАВАНЕ НА ГРАДОВЕТЕ ПО КАРТАТА
- CITYLIST
  - ФУНКЦИОНАЛНОСТИ ЗА ГРАДОВЕТЕ
- MAP
  - ФУНКЦИОНАЛНОСТИ ЗА КАРТАТА

# SINGLE RESPONSIBILITY PRINCIPLE (SRP)

## ПРИНЦИП ЗА ЕДИНСТВЕНА ОТГОВОРНОСТ

### ПРИМЕР...

- BANKACCOUNT
  - НОМЕР
  - БАЛАНС
  - ФУНКЦИОНАЛНОСТ ЗА ПРЕСМЯТАНЕ НА ЛИХВА
- ИМЕ НА ПРИТЕЖАТЕЛЯ
- ПРОМЯНА В ЛОГИКАТА НА ПРЕСМЯТАНЕ

# SINGLE RESPONSIBILITY PRINCIPLE (SRP)

## ПРИНЦИП ЗА ЕДИНСТВЕНА ОТГОВОРНОСТ

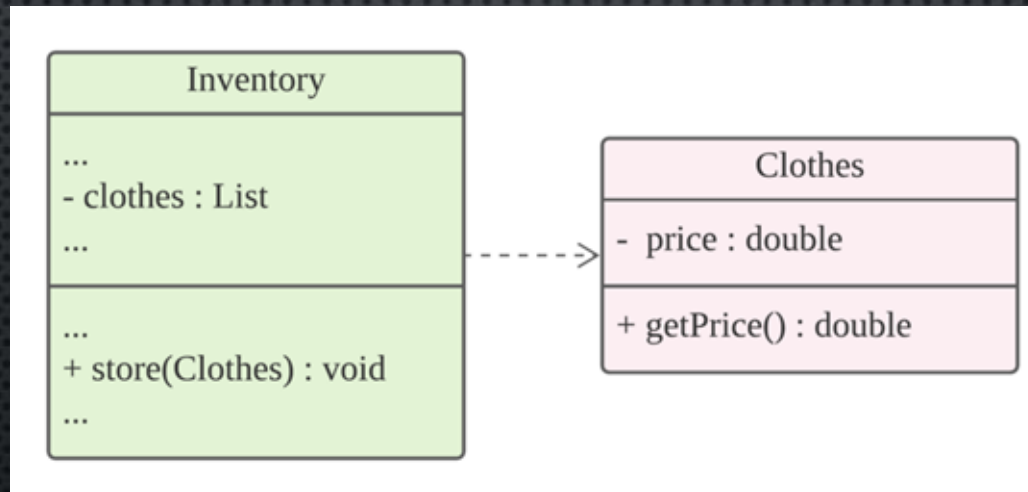
### ПРИМЕР...

- ORDERPROCESSOR
  - ПРОВЕРКА ДАЛИ КРЕДИТНАТА КАРТА Е ПРИЕТА –ФИНАНСИ.
  - ПРОВЕРКА ДАЛИ ПАРИТЕ СА ВЗЕТИ – ФИНАНСИ.
  - ПРОВЕРКА ДАЛИ АРТИКУЛЪТ Е НАЛИЧЕН – ИНВЕНТАР
  - ЗАЯВКА НА АРТИКУЛА ЗА РЕЗЕРВАЦИЯ.
  - ПРЕСМЯТАНЕ ВРЕМЕ ЗА ДОСТАВКА
  - ИЗПРАЩАНЕ НА ПОТВЪРЖДЕНИЕТО ПО ИМЕЙЛ НА КЛИЕНТА

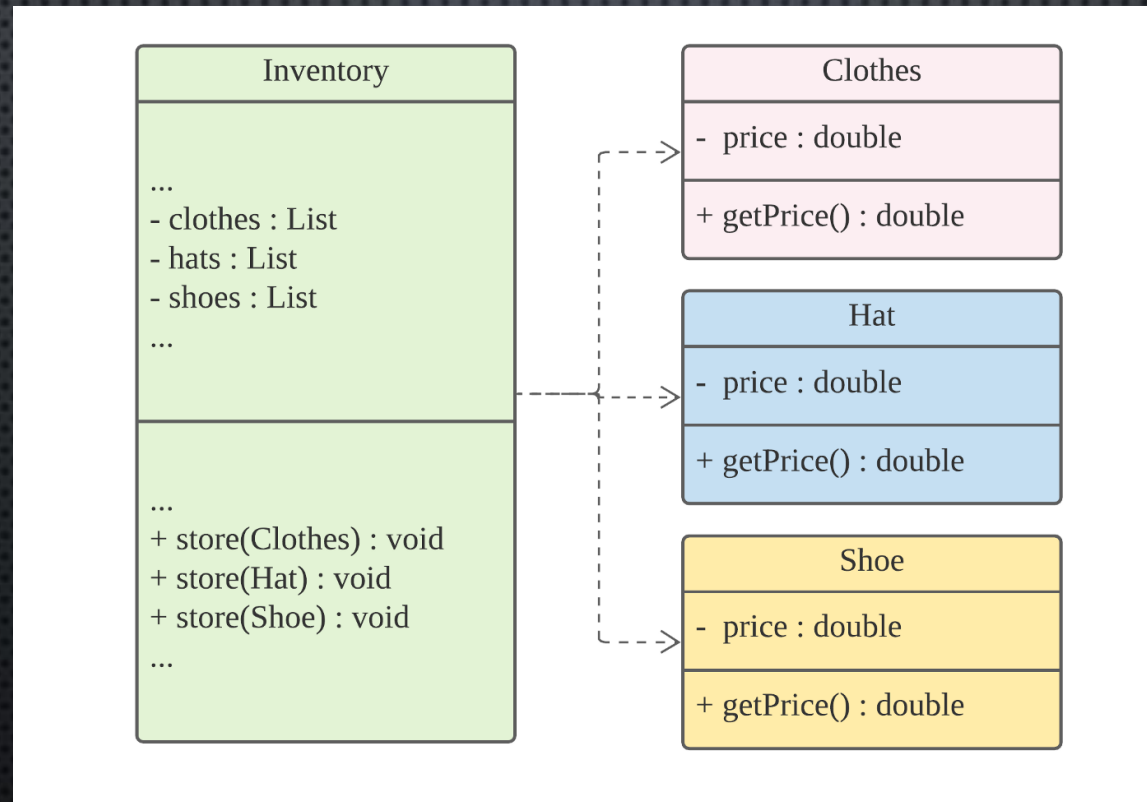
# OPEN CLOSED PRINCIPLE (ОСР) ПРИНЦИП ОТВОРЕН/ЗАТВОРЕН

- Един модул трябва да е отворен за разширение, но затворен за модификация
  - Промяна на това, което правят модулите, без да променяме изходния им код
  - Базира се на абстракция
    - Динамичен полиморфизъм ! - OVERRIDING
    - Статичен полиморфизъм ! - OVERLOADING

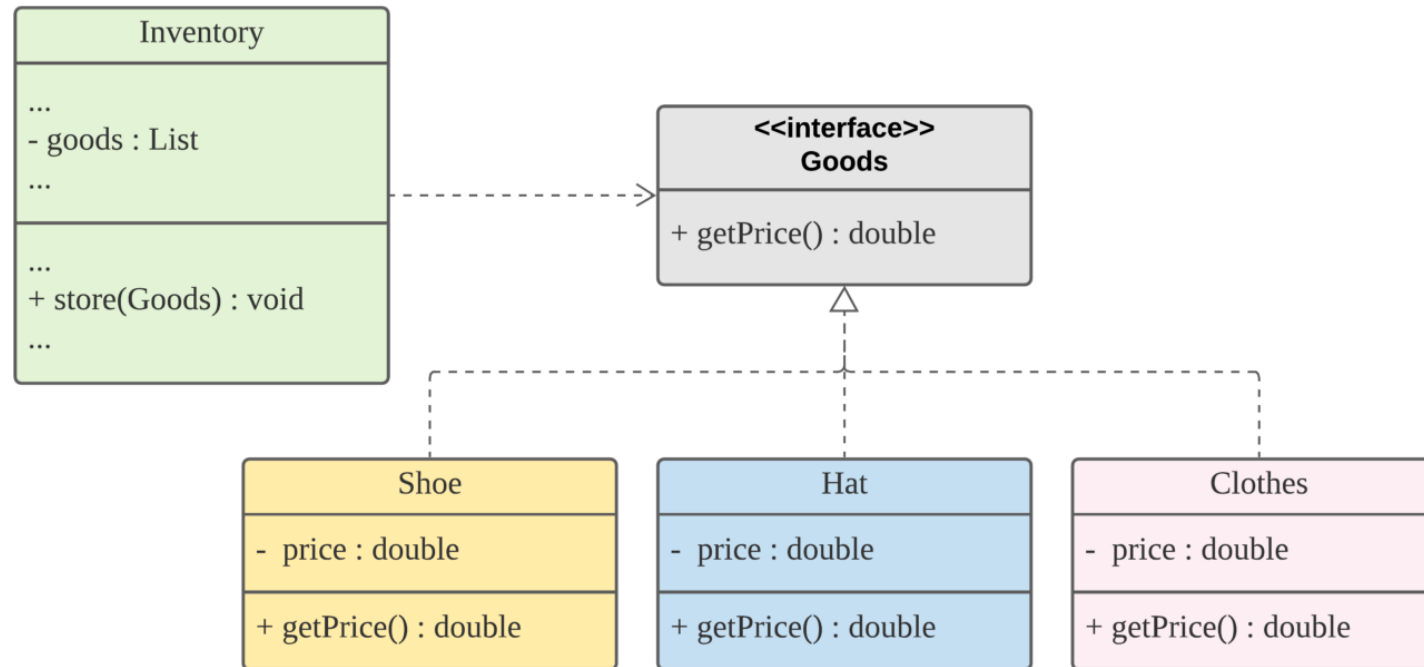
# OPEN CLOSED PRINCIPLE (ОСР) ПРИНЦИП ОТВОРЕН/ЗАТВОРЕН ПРИМЕР...



# OPEN CLOSED PRINCIPLE (ОСР) ПРИНЦИП ОТВОРЕН/ЗАТВОРЕН ПРИМЕР...



# OPEN CLOSED PRINCIPLE (ОСР) ПРИНЦИП ОТВОРЕН/ЗАТВОРЕН ПРИМЕР...



# LISKOV SUBSTITUTION PRINCIPLE ПРИНЦИП НА ЛИСКОВ ЗА ЗАМЕСТВАНЕ

- В КОМПЮТЪРНАТА ПРОГРАМА, АКО  $S$  Е ПОДТИП НА  $T$ , ТО ОБЕКТите ОТ ТИП  $T$  МОГАТ ДА БЪДАТ ЗАМЕНЕНИ ОТ ТЕЗИ ОТ ТИП  $S$ , БЕЗ ТОВА ДА НАМАЛЯВА ФУНКЦИОНАЛНОСТИТЕ НА ПРОГРАМАТА
  - ВСЕКИ НАСЛЕДНИК (ПОДТИП) ТРЯБВА ЛЕСНО ДА ЗАМЕНЯ ВСИЧКИТЕ СИ БАЗОВИ ТИПОВЕ
  - ПРОБЛЕМИ -> ПРОБЛЕМИ С ДИЗАЙНА

# LISKOV SUBSTITUTION PRINCIPLE

- КЛАСИЧЕСКИ НАРУШЕНИЯ НА ЛИСКОВ ПРИНЦИПА
  - НАРУШАВАНЕ НА ПОЛИМОРФИЗМА. — КЛАСЪТ НАСЛЕДНИК НЕ НАСЛЕДЯВА ВСИЧКИ ФУНКЦИОНАЛНОСТИ НА КЛАСА РОДИТЕЛ.
  - НАСЛЕДЕНИ МЕТОДИ ЗА МОДИФИЦИРАНЕ, КОИТО НЕ СА ИМПЛЕМЕНТИРАНИ.
  - НАСЛЕДЕНИК ПРЕМАХВА/СКРИВА ПОВЕДЕНИЕ ОТ БАЗОВИЯ КЛАС.

# LISKOV SUBSTITUTION PRINCIPLE ПРИНЦИП НА ЛИСКОВ ЗА ЗАМЕСТВАНЕ

- РЕШЕНИЯ
  - НЕ СЕ ИНТЕРЕСУВАМЕ ОТ ТИПА НА ОБЕКТА А ИЗИСКВАМЕ ОТ НЕГО ОПРЕДЕЛЕНА ФУНКЦИОНАЛНОСТ.
  - ИЗНАСЯНЕ НА ФУНКЦИОНАЛНОСТ В БАЗОВИЯ КЛАС ИЛИ ДОБАВЯНЕ НА НОВИ КЛАСОВЕ ЗА УСЪВЪРШЕНСТВАНЕ НА КЛАСОВАТА ЙЕРАРХИЯ

# LISKOV SUBSTITUTION PRINCIPLE ПРИНЦИП НА ЛИСКОВ ЗА ЗАМЕСТВАНЕ ПРИМЕР...

- КВАДРАТ НАСЛЕДЯВА ПРАВОЪГЪЛНИК
  - РАЗЛИКА В АТРИБУТИТЕ (РАЗЛИКА В ИМПЛЕМЕНТАЦИЯТА НА ФУНКЦИОНАЛНОСТ)
  - ХАК – ЛОША ПРАКТИКА
  - НАСЛЕДНИЦИ НА ФИГУРА

# INTERFACE SEGREGATION PRINCIPLE ПРИНЦИП ЗА РАЗДЕЛЯНЕ НА ИНТЕРФЕЙСИТЕ

- ДА НЕ СЕ ПРЕДОСТАВЯ ФУНКЦИОНАЛНОСТ, КОЯТО НЕ Е НЕОБХОДИМА
  - НАСЛЕДЯВАНЕ НА ИНТЕРФЕЙС

# INTERFACE SEGREGATION PRINCIPLE ПРИНЦИП ЗА РАЗДЕЛЯНЕ НА ИНТЕРФЕЙСИТЕ ПРИМЕР...

- ИНТЕРФЕЙС MEMBERSHIP
  - LOGIN
  - LOGOUT
  - REGISTER
  - FORGOTPASSWORD

# INTERFACE SEGREGATION PRINCIPLE ПРИНЦИП ЗА РАЗДЕЛЯНЕ НА ИНТЕРФЕЙСИТЕ ПРИМЕР...

- РЕШЕНИЕ?
  - LOGIN < - - - REGISTER
  - LOGIN, REGISTER

# DEPENDENCY INVERSION PRINCIPLE (DIP)

## ПРИНЦИП НА ОБРЪЩАНЕ НА ЗАВИСИМОСТИТЕ

- МОДУЛИТЕ НА ВИСОКО НИВО НЕ ТРЯБВА ДА ЗАВИСЯТ ОТ МОДУЛИТЕ НА НИСКО НИВО
  - И ДВЕТЕ ТРЯБВА ДА ЗАВИСЯТ ОТ АБСТРАКЦИИТЕ
- АБСТРАКЦИИТЕ НЕ ТРЯБВА ДА ЗАВИСЯТ ОТ ДЕТАЙЛИТЕ
  - ДЕТАЙЛИТЕ ТРЯБВА ДА ЗАВИСЯТ ОТ АБСТРАКЦИИТЕ
- ДА ИЗБЕГНЕ СВЪРЗАНОСТТА МЕЖДУ МОДУЛИТЕ, КАТО СЕ ПОЛЗВА АБСТРАКТЕН МОДУЛ

# DEPENDENCY INVERSION PRINCIPLE (DIP)

## ПРИНЦИП НА ОБРЪЩАНЕ НА ЗАВИСИМОСТИТЕ

### ПРИМЕР...

- УСЛУГА ЗА КЛИЕНТИ
  - ИЗВЛИЧАНЕ ОТ БАЗА (CUSTOMERDAO)— FINDBYID, FINDALL
  - ПРОБЛЕМ?
  - РЕШЕНИЕ?
    - ИНТЕРФЕЙС
      - ИМПЛЕМЕНТАЦИИ НА ИНТЕРФЕЙСА

# DEPENDENCY INVERSION PRINCIPLE (DIP)

## ПРИНЦИП НА ОБРЪЩАНЕ НА ЗАВИСИМОСТИТЕ

### ПРИМЕР...

```
INTERFACE CUSTOMERREPOSITORY {  
    CUSTOMER FINDById(LONG ID);  
    LIST<CUSTOMER> FINDAll();  
}
```

```
CLASS CUSTOMERSERVICE {  
    PRIVATE FINAL CUSTOMERREPOSITORY REPO;  
    CUSTOMERSERVICE(CUSTOMERREPOSITORY REPO) { THIS.REPO = REPO; }  
  
    CUSTOMER GET(LONG ID) { RETURN REPO.FINDById(ID); }  
    LIST<CUSTOMER> ALL() { RETURN REPO.FINDAll(); }  
}
```